

Apunte Selectivo Progcomp Uchile / ICPC

Felipe Colli Olea

13 de agosto de 2026

Índice

1. Plantillas Base	2
1.1. Template Básico (con casos de prueba)	2
2. Estructuras de Datos	2
2.1. Segment Tree (Min Query)	2
2.2. Segment Tree (Max Query)	3
3. Grafos (Tu Salvador)	3
3.1. Búsqueda en Profundidad (DFS)	3
3.2. Búsqueda en Anchura (BFS)	3
3.3. Grafos Implícitos (Grillas y Laberintos)	4
3.4. Dijkstra (Camino Mínimo CON peso positivo)	4
4. Strings y Textos	5
4.1. KMP Prefix Function	5
4.2. Partir (Split) un String por Delimitadores	5
5. Programación Dinámica (DP)	6
5.1. Distancia de Edición (Edit Distance)	6
5.2. Subsecuencia Común Más Larga (LCS)	6
6. Matemáticas / Utilidades	6
6.1. Generar todas las permutaciones	6
7. Estructuras "Mágicasz Fáciles	7
7.1. Ordered Set (PBDS)	7
7.2. Fenwick Tree (BIT)	7
7.3. Disjoint Set Union (DSU)	7
8. Matemáticas	8
8.1. Criba de Eratóstenes (Sieve)	8
8.2. Exponenciación Binaria y Modulo	8

1. Plantillas Base

1.1. Template Básico (con casos de prueba)

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 typedef long long ll;
5 typedef string str;
6 #define vec vector
7
8 int main() {
9     ios_base::sync_with_stdio(0); cin.tie(0);
10    int t;
11    cin >> t;
12    while (t--) {
13        // Logica aqui
14    }
15    return 0;
16 }
```

2. Estructuras de Datos

2.1. Segment Tree (Min Query)

```
1 struct SegmentTree {
2     int n;
3     vec<ll> Tree;
4
5     SegmentTree(vec<ll>& a) {
6         n = a.size();
7         Tree.resize(4 * n);
8         build(a, 0, 0, n - 1);
9     }
10
11    void build(vec<ll>& a, int nodo, int izq, int der) {
12        if (izq == der) {
13            Tree[nodo] = a[izq];
14            return;
15        }
16        int mid = izq + (der - izq) / 2;
17        build(a, 2 * nodo + 1, izq, mid);
18        build(a, 2 * nodo + 2, mid + 1, der);
19        Tree[nodo] = min(Tree[2 * nodo + 1], Tree[2 * nodo + 2]);
20    }
21
22    ll q(int l, int r) {
23        return query(0, 0, n - 1, l, r);
24    }
25
26    ll query(int nodo, int izq, int der, int l, int r) {
27        if (r < izq || l > der) return LLONG_MAX;
28        if (l <= izq && der <= r) return Tree[nodo];
29        int mid = izq + (der - izq) / 2;
30        return min(query(2 * nodo + 1, izq, mid, l, r),
31                    query(2 * nodo + 2, mid + 1, der, l, r));
32    }
```

```

32     }
33 };

```

2.2. Segment Tree (Max Query)

Para Max Query, usa el mismo código de arriba, pero reemplaza `min` por `max` y el retorno fuera de rango por `LLONG_MIN`.

3. Grafos (Tu Salvador)

Nota: Si los grafos no son lo tuyo, solo copia y adapta. Están optimizados y resumidos respecto al PDF para que no falles en la implementación.

3.1. Búsqueda en Profundidad (DFS)

Ideal para contar componentes conexas, detectar ciclos, o simplemente recorrer.

```

1 vector<int> visitados;
2 vector<vector<int>> AdjList;
3
4 void dfs(int u) {
5     visitados[u] = 1;
6     for (int v : AdjList[u]) {
7         if (!visitados[v]) {
8             dfs(v);
9         }
10    }
11 }
12 // En main(): visitados.assign(N, 0); AdjList.assign(N, vector<int>());

```

3.2. Búsqueda en Anchura (BFS)

Encuentra el **camino mínimo en grafos SIN peso** (distancia, cantidad de saltos).

```

1 vector<int> dist;
2 vector<vector<int>> AdjList;
3
4 void bfs(int inicio) {
5     dist[inicio] = 0;
6     queue<int> q;
7     q.push(inicio);
8
9     while (!q.empty()) {
10        int u = q.front();
11        q.pop();
12
13        for (int v : AdjList[u]) {
14            if (dist[v] == -1) { // -1 significa no visitado
15                dist[v] = dist[u] + 1;
16                q.push(v);
17            }
18        }
19    }
20 }
21 // En main(): dist.assign(N, -1);

```

3.3. Grafos Implícitos (Grillas y Laberintos)

Cuando el problema es un tablero/matriz, no construimos un `AdjList`. Usamos los arreglos `dr` y `dc` para movernos (Arriba, Abajo, Izq, Der).

```
1 int MAX_FILAS, MAX_COLUMNAS;
2 vector<vector<char>> grilla;
3
4 // Movimientos: S, SE, E, NE, N, NW, W, SW (si piden 8 direcciones)
5 // Para 4 direcciones usa solo los valores correspondientes.
6 int dr[] = { 1, 1, 0, -1, -1, -1, 0, 1 };
7 int dc[] = { 0, 1, 1, 1, 0, -1, -1, -1 };
8
9 bool limites(int f, int c) {
10     return (f >= 0 && f < MAX_FILAS && c >= 0 && c < MAX_COLUMNAS);
11 }
12
13 // Ejemplo explorando con BFS
14 void bfs_grilla(int start_f, int start_c) {
15     queue<pair<int, int>> q;
16     q.push({start_f, start_c});
17
18     while (!q.empty()) {
19         auto [f, c] = q.front(); q.pop();
20
21         for (int i = 0; i < 8; ++i) { // 0 i < 4 si son 4 direcciones
22             int nf = f + dr[i];
23             int nc = c + dc[i];
24             if (limites(nf, nc) /* && condicion_extra */) {
25                 // Procesar vecino nf, nc
26                 q.push({nf, nc});
27             }
28         }
29     }
30 }
```

3.4. Dijkstra (Camino Mínimo CON peso positivo)

Encuentra la ruta más barata. ¡Ojo! Acá se usa una `priority_queue` en reversa (de menor a mayor).

```
1 vector<long long> dist;
2 // AdjList guarda {nodo_destino, peso_arista}
3 vector<vector<pair<int, int>>> AdjList;
4 const long long INF = LLONG_MAX;
5
6 void dijkstra(int inicio) {
7     dist[inicio] = 0;
8     // La cola guarda {distancia_acumulada, nodo_actual}
9     priority_queue<pair<long long, int>,
10                   vector<pair<long long, int>>,
11                   greater<pair<long long, int>>> pq;
12
13     pq.push({0, inicio});
14
15     while (!pq.empty()) {
16         auto [d_actual, u] = pq.top();
17         pq.pop();
```

```

18
19     // Si encontramos un camino peor guardado en la cola, lo ignoramos
20     if (d_actual > dist[u]) continue;
21
22     for (auto edge : AdjList[u]) {
23         int v = edge.first;
24         int peso = edge.second;
25
26         if (dist[u] + peso < dist[v]) {
27             dist[v] = dist[u] + peso;
28             pq.push({dist[v], v});
29         }
30     }
31 }
32 }
33 // En main(): dist.assign(N, INF);

```

4. Strings y Textos

4.1. KMP Prefix Function

Encuentra ocurrencias de patrones en tiempo lineal.

```

1 vector<ll> prefix_function(const string& s) {
2     int n = s.size();
3     vector<ll> pi(n, 0);
4     for (int i = 1; i < n; i++) {
5         int j = pi[i - 1];
6         while (j > 0 && s[i] != s[j])
7             j = pi[j - 1];
8         if (s[i] == s[j])
9             j++;
10        pi[i] = j;
11    }
12    return pi;
13 }

```

4.2. Partir (Split) un String por Delimitadores

Usando la librería de C (<cstring>) para separar oraciones por espacios o comas.

```

1 string texto = "Este texto, tiene comas!";
2 char* token = strtok(&texto[0], " ,!"); // Delimitadores: espacio, coma,
    exclamacion
3
4 vector<string> palabras;
5 while (token != NULL) {
6     palabras.push_back(token);
7     token = strtok(NULL, " ,!");
8 }

```

5. Programación Dinámica (DP)

5.1. Distancia de Edición (Edit Distance)

Mínimo costo de conversiones (insertar, borrar, reemplazar) para igualar dos strings. Costo unitario = 1.

```
1 int min3(int x, int y, int z) { return min(min(x, y), z); }
2
3 int edit_distance(string str1, string str2) {
4     int m = str1.size(), n = str2.size();
5     vector<vector<int>> dp(m + 1, vector<int>(n + 1));
6     for (int i = 0; i <= m; i++) {
7         for (int j = 0; j <= n; j++) {
8             if (i == 0) dp[i][j] = j;
9             else if (j == 0) dp[i][j] = i;
10            else if (str1[i - 1] == str2[j - 1]) dp[i][j] = dp[i - 1][j - 1];
11            else dp[i][j] = 1 + min3(dp[i][j - 1],      // Insertar
12                                   dp[i - 1][j],      // Remover
13                                   dp[i - 1][j - 1]); // Reemplazar
14        }
15    }
16    return dp[m][n];
17 }
```

5.2. Subsecuencia Común Más Larga (LCS)

Encuentra el tamaño de la cadena común más larga (no necesitan ser continuas).

```
1 int LCS(string str1, string str2) {
2     int m = str1.size(), n = str2.size();
3     vector<vector<int>> L(m + 1, vector<int>(n + 1, 0));
4     for (int i = 0; i <= m; i++) {
5         for (int j = 0; j <= n; j++) {
6             if (i == 0 || j == 0) L[i][j] = 0;
7             else if (str1[i - 1] == str2[j - 1])
8                 L[i][j] = L[i - 1][j - 1] + 1;
9             else
10                L[i][j] = max(L[i - 1][j], L[i][j - 1]);
11        }
12    }
13    return L[m][n];
14 }
```

6. Matemáticas / Utilidades

6.1. Generar todas las permutaciones

```
1 string s = "CBA";
2 sort(s.begin(), s.end()); // IMPORTANTISIMO: Siempre ordenar antes
3 vector<string> salida;
4 do {
5     salida.push_back(s);
6 } while (next_permutation(s.begin(), s.end()));
```

7. Estructuras "Mágicas Fáciles"

7.1. Ordered Set (PBDS)

Es un set normal, pero tiene dos funciones extra: `find_by_order(k)` (devuelve iterador al k-ésimo elemento, 0-indexado) y `order_of_key(x)` (cuenta cuántos elementos son estrictamente menores a x).

```
1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4
5 template <typename T>
6 using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
7     tree_order_statistics_node_update>;
8
9 // Uso:
10 // ordered_set<int> st;
11 // st.insert(5); st.insert(2);
12 // cout << *st.find_by_order(0); // Imprime 2
13 // cout << st.order_of_key(5); // Imprime 1 (solo el 2 es menor)
```

7.2. Fenwick Tree (BIT)

Suma en rangos y actualización de un punto en $O(\log n)$. Más corto que SegTree. (1-indexado internamente).

```
1 struct BIT {
2     int n; vector<ll> bit;
3     BIT(int n) : n(n), bit(n + 1, 0) {}
4     void add(int idx, ll val) {
5         for (++idx; idx <= n; idx += idx & -idx) bit[idx] += val;
6     }
7     ll query(int idx) { // suma de [0, idx]
8         ll ret = 0;
9         for (++idx; idx > 0; idx -= idx & -idx) ret += bit[idx];
10        return ret;
11    }
12    ll query(int l, int r) { return query(r) - query(l - 1); }
13};
```

7.3. Disjoint Set Union (DSU)

Agrupar elementos y verificar si dos elementos están en el mismo grupo. Excelente para problemas de conectividad o Kruskal (MST).

```
1 struct DSU {
2     vector<int> p, sz;
3     DSU(int n) {
4         p.resize(n); iota(p.begin(), p.end(), 0);
5         sz.assign(n, 1);
6     }
7     int find(int i) {
8         if (p[i] == i) return i;
9         return p[i] = find(p[i]); // Path compression
10    }
11    bool unite(int i, int j) {
```

```

12     int root_i = find(i), root_j = find(j);
13     if (root_i == root_j) return false;
14     if (sz[root_i] < sz[root_j]) swap(root_i, root_j);
15     p[root_j] = root_i;
16     sz[root_i] += sz[root_j];
17     return true;
18 }
19 };

```

8. Matemáticas

8.1. Criba de Eratóstenes (Sieve)

Encuentra todos los primos hasta N y el factor primo más pequeño (SPF) de cada número.

```

1 const int MAXN = 1e6 + 5; // Cambiar segun problema
2 int spf[MAXN]; // Smallest Prime Factor
3 vector<int> primes;
4
5 void sieve() {
6     for (int i = 2; i < MAXN; i++) spf[i] = i;
7     for (int i = 2; i * i < MAXN; i++) {
8         if (spf[i] == i) {
9             for (int j = i * i; j < MAXN; j += i)
10                 if (spf[j] == j) spf[j] = i;
11         }
12     }
13     for (int i = 2; i < MAXN; i++)
14         if (spf[i] == i) primes.push_back(i);
15 }
16 // factorizar rapido: while(n > 1) { factors.push_back(spf[n]); n /= spf[n]; }

```

8.2. Exponenciación Binaria y Modulo

Calcula a^b (mód m) en $O(\log b)$.

```

1 ll binpow(ll a, ll b, ll m) {
2     a %= m;
3     ll res = 1;
4     while (b > 0) {
5         if (b & 1) res = res * a % m;
6         a = a * a % m;
7         b >>= 1;
8     }
9     return res;
10 }
11
12 // Inverso modular de A mod M (Solo funciona si M es primo)
13 ll modInverse(ll a, ll m) {
14     return binpow(a, m - 2, m);
15 }
16 // Division modular (a / b) mod m
17 ll modDivide(ll a, ll b, ll m) {
18     return (a % m) * modInverse(b, m) % m;
19 }

```